



The Role of Data Structures in Atmospheric Modeling:

Using an Eulerian Octree to Model Heat Dispersion

Emma Krebs

TABLE OF CONTENTS

Data Structures and Atmospheric Modeling

Why do we care about how we store data
for projects like atmospheric modeling?

01

What is an Eulerian Octree?

What is Eulerian and what is an
octree? What makes it good for
these types of simulations?

02

Overview of Code

Summary of the code and
Important Function Definitions

03

Resulting Animation

Here you could describe
the topic of the section

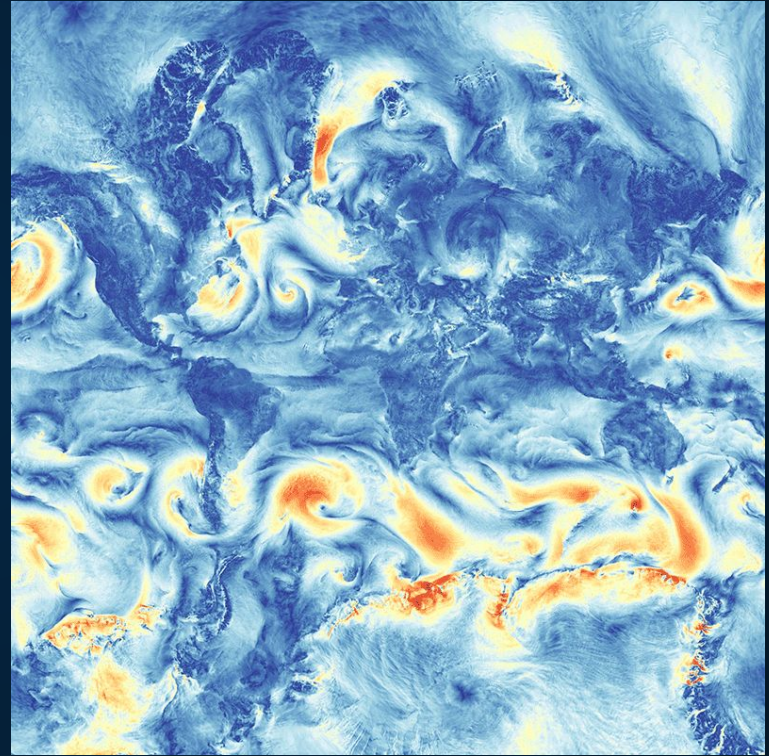
04

Data Structures and Atmospheric Modeling

Atmospheric Models are complex simulations of millions of data points.

- Temperature
- Wind Shear
- Vorticity
- Pressure
- Etc.

Working in 3D increases this complexity, so we need smart data structures to be able to run these simulations.



Max Planck Institute wind speed near Earth's surface with 1km resolution.

Data Structures and Atmospheric Modeling

Although grids and arrays can outperform some data structures with less information to keep track of, they slow down because every individual point in the grid is saved to that structure.

For 3D space, the number of points to keep track of is N^3 . This isn't the best use of our memory if we are only interested in regions of activity (ie, storm formation, intense wind shear, etc)

We need a data structure that stores all of this information, isn't computationally taxing, and can adapt to activity.

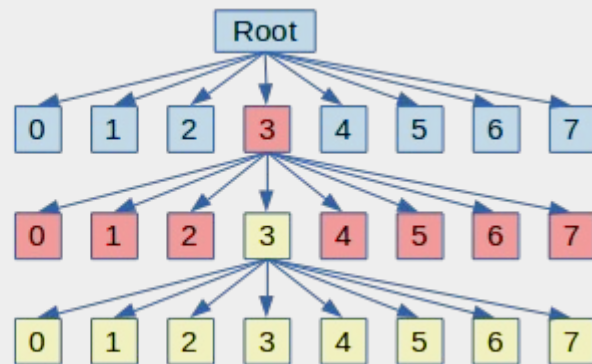
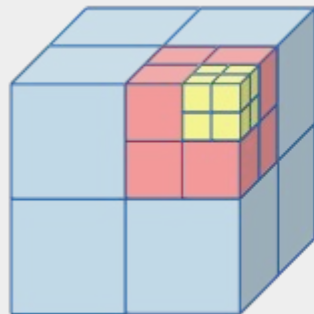
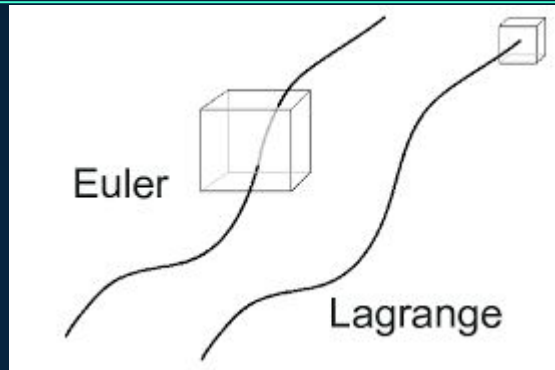
Introducing..... **THE OCTREE!!!!!!!!!!!!!!**

What is a Eulerian Octree?

We have two new terms to define:

Eulerian: One of two perspectives we can have when coding → Care about what moves through our region of interest. No individual tracking.

Octree: A tree structure that partitions 3D space by recursively dividing it into 8 children.

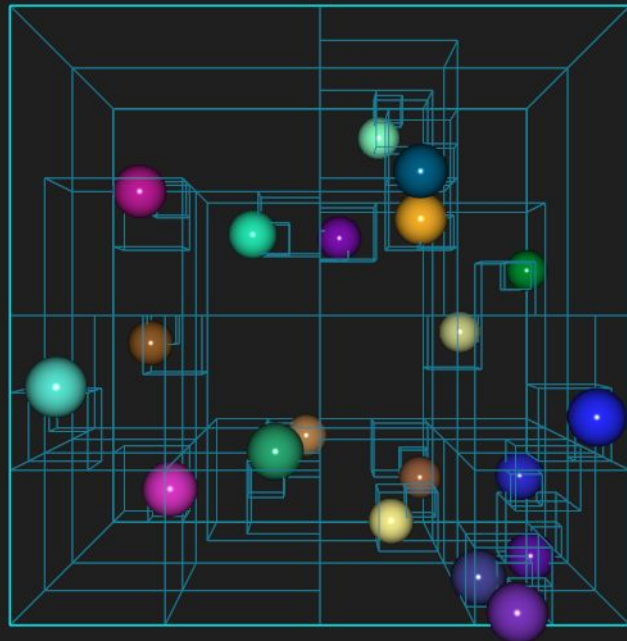


Why are Octrees good for 3D Simulations?

Eulerian Octrees are an adaptive mesh, meaning they change depending on activity in a region. This is extremely good for spatial checks.

The green ball wants to see what is around it. Check every single position for a ball, or just check its neighbors?

- Only store memory for neighbors.
- $O(\log n)$ vs $O(n)$ (Neighbor lookup)
- Either check 20 nodes or 1 million nodes...



Where did you hear of octrees? (Pathfinding and DLA)



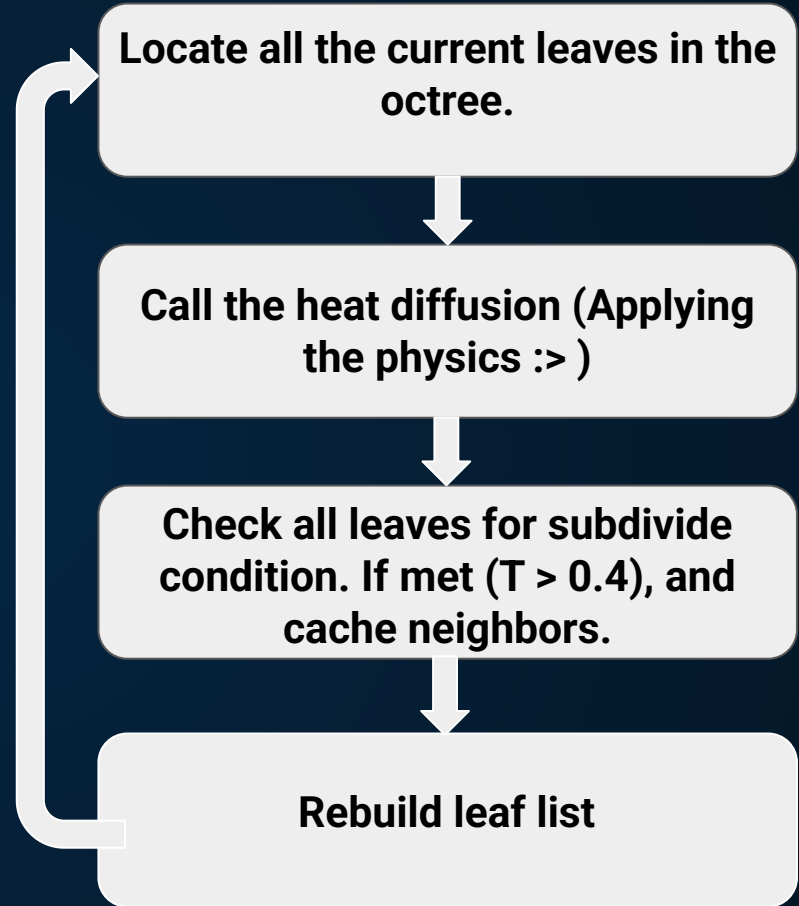
Also used in robotics LiDAR for self-driving robots, minecraft raytracing. and more!
Tried to use it for DLA initially.

Code Summary

We will be testing heat diffusion as a pseudo atmospheric model (also because I ran out of time for the other functions...).

Start by creating a root node to start our octree, located at the center of our 3D space.

Then, loop through time step with the following steps:



class Node (Object)

This is the main storage unit for our program. Data stored for a cell are:

- Cell center
- Cell size
- Current node depth
- Whether it is a leaf, aka no children.
- Temperature
- Neighbors
- (AND MORE!!)

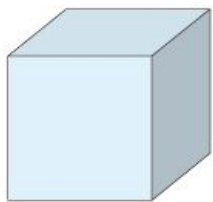
```
def __init__(self, center, size, depth):
    self.center = center
    self.size = size
    self.depth = depth
    self.leaf = True
    self.children = [None, None, None, None, None, None, None, None] # Produces 8 children for subdivision

    # Fluid dynamic variables
    self.u = 0 # Velocity for X axis
    self.v = 0 # Velocity for Y axis
    self.w = 0 # Velocity for Z axis
    self.omega = [0, 0, 0] # Vorticity for x-, y-, z- directions
    self.T = 0 # Temperature

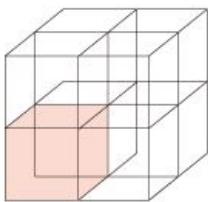
    # Calculate the neighbors after each rebuilding of leaf list. Includes six neighbors on each cell's face.
    # This is to shorten number of loops we have to go through to calculate temperature gradients.
    # Only needs to be updated if we subdivide/collapse the octree
    self.neighbors = {
        'xm': None, # all labels of m means minus direction. So this is face on the negative x side
        'xp': None, # all labels of p means positive direction. So this is face on the positive x side
        'ym': None,
        'yp': None,
        'zm': None,
        'zp': None,
    }
```

subdivide

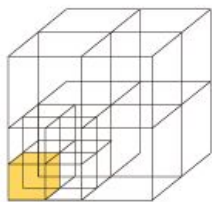
When this function is called, it subdivides the space into 8 children. Children inherit temp.



Refinement
Level 1



Refinement
Level 2



Refinement
Level 3

```
if node.leaf != True or node.depth >= max_depth:  
    return
```

```
node.leaf = False # It is no longer a leaf case. It has children now!!!  
depth = node.depth + 1 # Update depth  
child_size = node.size / 2
```

```
offsets = [-child_size/2, child_size/2] # Need to offset center for grid so it is in middle of box.
```

```
child_index = 0  
for dx in offsets:  
    for dy in offsets:  
        for dz in offsets:  
            center_x = node.center[0] + dx  
            center_y = node.center[1] + dy  
            center_z = node.center[2] + dz  
  
            child = Node([center_x, center_y, center_z], child_size, depth)  
  
            # The parents parameters will become the children's  
            child.u = node.u  
            child.v = node.v  
            child.w = node.w  
            child.omega = node.omega.copy() # Bc it is array  
            child.T = node.T  
  
            node.children[child_index] = child  
            child_index += 1
```

find_node

Finds node given a point by using color quantization program. Stores values as binary that change for each coordinate we check. Recursive. Show the example Emma...

Any combination results in a number 0-7. Gives us our index!

```
# From the wiki, we can use the color quantization program which determines the child node
# via the formula  $4r + 2g + b$ , but here instead of red, green, and blue we can use
# our positive and negative 3 directions. Thus
node = root

value = 0
# This will loop until it finds the leaf node to extract the particles
while node.leaf != True:
    value = 0

    if point[0] >= node.center[0]:
        value |= 4
    if point[1] >= node.center[1]:
        value |= 2
    if point[2] >= node.center[2]:
        value |= 1

    # Example: Say we said yes to all three if statements. Then we have 7 and that represents
    # our positive quadrant for this center.

    found_node = node.children[value]

    if found_node is None:
        return node, value
    node = found_node

return node, value
```

get_leaves

Grabs the leaves of a node. If given root finds all the leaves in the octree.

```
leaves = []

if node.leaf == True:
    return [node]

for child in node.children:
    if child != None:
        leaves.extend(get_leaves(child))

return leaves
```

get_neighbors

If given a node, it will probe surrounding six directions that touch the face of the cell. Creates a dictionary inside of the given node for easy access to its neighbors.

```
# Look for surrounding neighbors

x, y, z = node.center

# All the same distance, so use dx for dx, dy, dz
dx = node.size * 0.51

neighbors = {} # Dictionary of neighbors

probes = {
    'xm': [x - dx, y, z],
    'xp': [x + dx, y, z],
    'ym': [x, y - dx, z],
    'yp': [x, y + dx, z],
    'zm': [x, y, z - dx],
    'zp': [x, y, z + dx]
}

for key, probe in probes.items():
    neighbor, _ = find_node(root, probe)

    if neighbor is not None:
        neighbors[key] = neighbor
    else:
        neighbors[key] = None

return neighbors
```

Diffusion

Stores all the temperatures of neighbors. Calculates the difference between the neighbor temperature and the current node's temperature, and multiplies it by our diffusion rate alpha. Causes:

- Higher temperature to decrease each time step
- Lower temperature to increase

```
new_T = {}

for node in leaves:
    neighbor_temps = []

    # Use the cached neighbors instead of looping through all leaves
    for key in ['xm', 'xp', 'ym', 'yp', 'zm', 'zp']:
        nb = node.neighbors.get(key)
        if nb:
            neighbor_temps.append(nb.T)

    if neighbor_temps:
        avg = sum(neighbor_temps) / len(neighbor_temps)
        new_T[id(node)] = node.T + alpha * (avg - node.T)
    else:
        new_T[id(node)] = node.T

for node in leaves:
    node.T = new_T[id(node)]
```


Depth of the Octree

Before we see the final results, note that the depth greatly impacts the speed of the heat diffusion and resolution of the image:

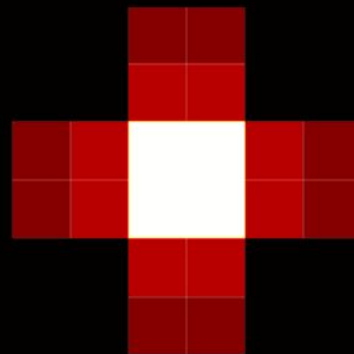
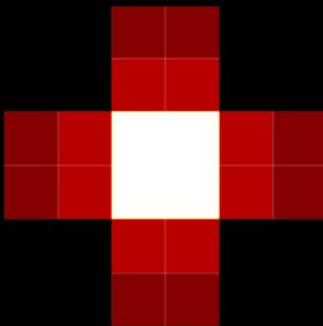
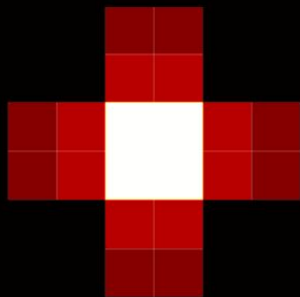
Higher Max Depth → Better Resolution
→ Slower diffusion
→ More computation power

Consider the image on the right.



Lower polynomial count vs high polynomial count for gaming.
Collisions/Visual vs Computation

Resulting Animations

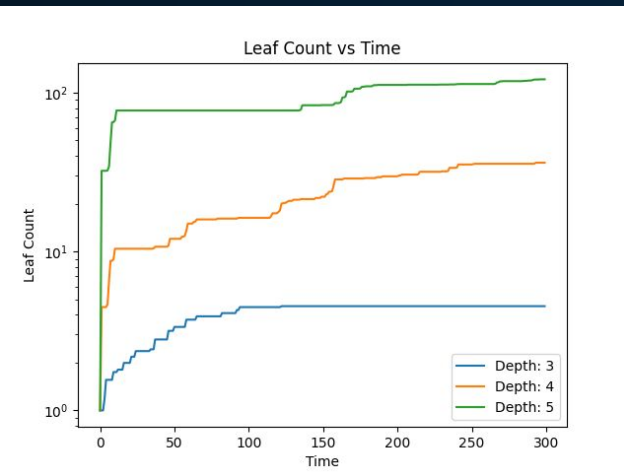


Max Number of Subdivisions: 3

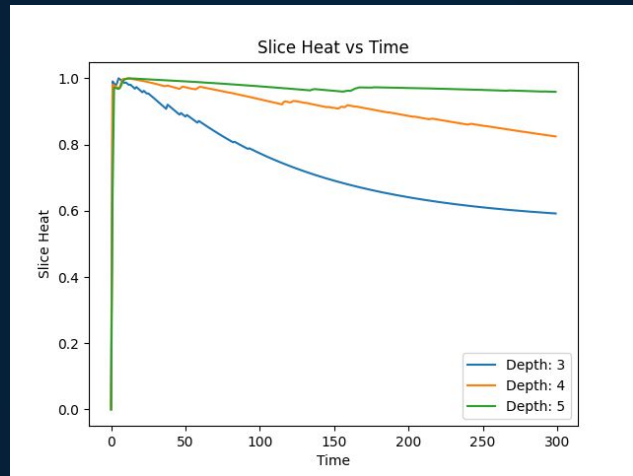
Max Number of Subdivisions: 4

Max Number of Subdivisions: 5

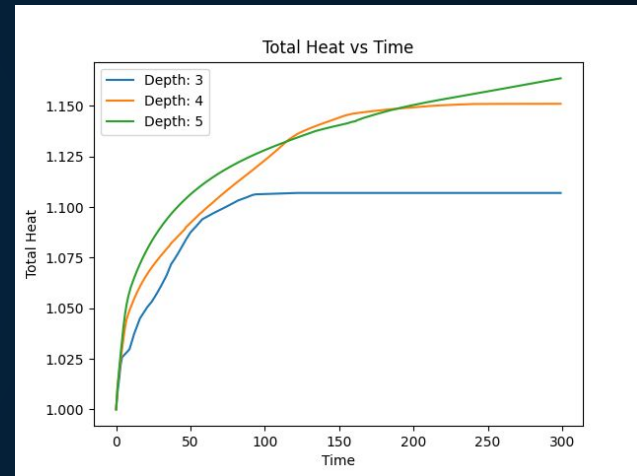
Looking at these Structures Closer



Leaves vs Time



Heat of Slice vs Time
(Temperature * size²)



Total Heat vs Time
(Temperature * size³)

Deeper octrees have more leaves, smoother resolutions, and different temperature diffusion behavior because refinement changes how many cells are participating in diffusion.

Conclusions

We can see that computational modeling of complex systems can get difficult fast. Incorporating an efficient and useful data structure for your code can make or break your simulation, especially when working with millions of data points.

The octree provides a unique structure to minimize computation time while providing a quick neighbor lookup structure for heat diffusion.

The deeper the octree the more leaves and thus better resolution we have. However, this worsens computation time.

Sources

PHOTOS:

-<https://mpimet.mpg.de/en/research/modeling> (Max Planck)

-<https://geidav.wordpress.com/2014/07/18/advanced-octrees-1-preliminaries-insertion-strategies-and-max-tree-depth/> (Octree)

-https://www.researchgate.net/figure/The-Eulerian-and-Lagrangian-ways-to-describe-the-flow_fig3_265103109 (Eulerian)

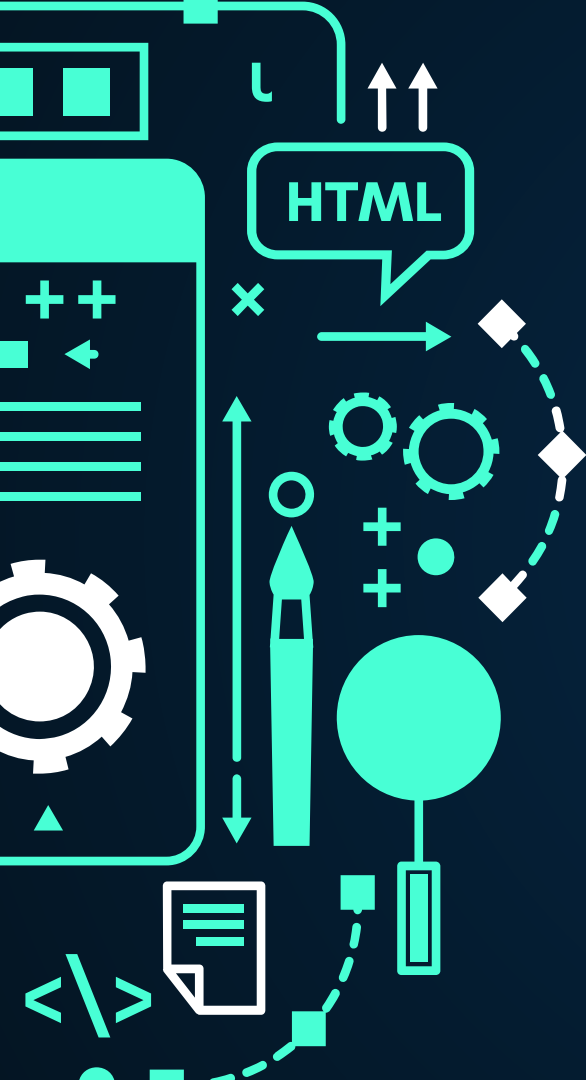
<https://doc.magnum.graphics/magnum/examples-octree.html> (Octree))

-<https://docs.o3de.org/docs/user-guide/gems/reference/kythera-ai/octree-generation/> (Octree tree)

-<https://progameguides.com/lethal-company/how-to-deal-with-forest-keeper-in-lethal-company/> (Lethal Company)

-<https://3dstudio.co/3d-modeling/low-and-high-poly-modeling/> (Trees)





THANKS!

Does anyone have any question?